

「柚子集點大冒險」楓之谷風格介面與音效實作指南 (UI/UX Specification)

本文件用途：供 Claude Code 於開發「M3 戰鬥系統」、「M4 英雄商店」以及「首頁 HUD 介面」時作為最高美術與音效技術指南。所有動畫轉移時間、CSS 特效參數、SVG 分層以及 Web Audio API 合成公式，均在此進行標準化規範。

1. 🎨 視覺系統與響應式版面規範 (Visual & Layout System)

為了在手機優先的 Web App (PWA) 上完美呈現 2D 賽璐珞 (Cel-shading) 風格，整體佈局與色彩需嚴格遵守以下規格：

1.1 楓之谷動態天空森林背景 (CSS Canvas Backdrop)

- 背景色盤 (Sky-to-Grass Gradient):
background: linear-gradient(to bottom, #80deea 0%, #e0f7fa 40%, #fff9c4 70%, #c8e6c9 100%);

這代表了從「晴空藍 → 暖晨黃 → 冒險綠地」的立體漸層。
- 動態漂浮雲朵 (Parallax Clouds):
使用不同速度的純白半透明圓角區塊作無限水平循環，製造 2D 捲軸層次感：
 - 雲朵 A: 寬 112px、高 32px，動畫週期 25s，毛玻璃濾鏡 blur(1px)。
 - 雲朵 B: 寬 160px、高 40px，動畫週期 35s，延遲 -10s。

1.2 楓之谷經典 3D 漸層字體 (MapleStory Damage Numbers)

這是重現「打擊爽快感」的靈魂所在。所有彈出的傷害文字必須套用以下 CSS：

- 字型設定: font-family: 'Fredoka One', Impact, sans-serif (確保圓潤厚重)。
- 3D 重黑描邊效果: 使用多重 text-shadow 來實現乾淨的純黑立體邊緣 (而非模糊的發光效果):
text-shadow:
-3px -3px 0 #000,
3px -3px 0 #000,
-3px 3px 0 #000,
3px 3px 0 #000,
4px 4px 0px rgba(0,0,0,0.7);
- 金黃色漸層填滿 (Gold Gradient):
background: linear-gradient(to bottom, #ffffff 0%, #fff176 30%, #f57f17 70%, #e65100 100%);
-webkit-background-clip: text;

-webkit-text-fill-color: transparent;

2. 🦸 2D 賽璐珞角色 SVG 分層規格 (Spine-like SVG Rigging)

所有英雄與怪獸不使用靜態圖片，而是使用 **SVG** 向量標籤進行元件化封裝，方便 Claude Code 以純 CSS/JS 控制其關節、武器揮擊角度與表情。

2.1 英雄 SVG 骨架分層 (以小勇者阿光為例)

每個英雄 SVG 視窗大小固定為 `viewBox="0 0 120 120"`，內部圖層由底至頂 (Z-index) 排序如下：

1. 左手臂與魔法球圖層：`<g>` 包含法力球半透明發光濾鏡。
2. 身體與飾品圖層：衣服背部的點心背包 `<circle>` → 衣服主體 `<rect rx="8">` → 腰帶。
3. 短腿與鞋子圖層：左右大腿 `<rect>` → 橢圓形小鞋 `<ellipse cx="..." cy="107">`。
4. 頭部與膚色圖層：巨大圓形頭部 `<circle cx="60" cy="48" r="26" fill="#D4A373">`。
5. 表情圖層：雙頰圓形紅暈 `<ellipse fill="#ff8a80" opacity="0.8">` → 雙眼大黑豆 `<ellipse>` 與白色反光點 → 嘴巴弧線。
6. 劉海與頭盔圖層：蓬鬆褐色劉海 `<path>` → 平底鍋頭盔 `<path>` → 手柄。
7. 右手臂與武器圖層：手臂 `<rect>` → 新手木劍 `<path>`。此組件必須設定 ID `#hero-weapon`，以便在蓄力時做旋轉過渡。

2.2 怪獸 SVG 狀態抽換規格 (表情與教養聯結)

怪獸 SVG 大小固定為 `viewBox="0 0 100 100"`，預設水平翻轉 (`scaleX(-1)`)。

- 正常狀態 (Normal Group ID: `#monster-eyes-normal`):
包含兩組黑色的惡作劇大眼 `<circle>`、嘴巴 `<path>` (邪惡歪嘴笑)。
- 大哭狀態 (Crying Group ID: `#monster-eyes-crying` & `#monster-mouth-crying`):
 - 眼睛變為倒 U 型哭泣線條：`<path stroke-linecap="round" fill="none">`。
 - 動態流淚粒子 (淚流雨) `#tear-stream`: 兩道亮藍色 (`#00b0ff`) 的水滴矩陣，綁定 CSS 位移與縮放動畫，不斷向下滴落。
 - 嘴巴變為向下垮的倒 V 型。
- 觸發邏輯：當怪獸當前 HP 佔最大 HP 比例 $\leq 30\%$ 時，程式必須移除 `#monster-eyes-normal` 與 `#monster-mouth` 的 hidden class，並顯示大哭組件。

3. 🗡️ 動作狀態機與戰鬥特效時間軸 (State Machine)

戰鬥系統 (M3) 點擊「發動攻擊」時，必須嚴格按照以下毫秒級 (ms) 時間軸驅動角色與場景特效：

時間點 (t)	英雄動作 (Hero)	怪獸/場景動態 (Monster/Scene)	播放音效 (Audio)
0.0s	切換為 anim-prep (蓄力)	怪獸正常嘲諷待機	-
0.4s	切換為 anim-slash (向前突進)	-	播放 揮刀聲 (Swing)
0.55s	-	怪獸受擊 (Hurt)、閃白、螢幕震動	播放 擊中爆破聲 (Hit)
-	-	彈出 3D 傷害數字與 Critical! 標籤	-

└─ 0.60s - 若 HP <= 30%, 觸發 Crying (大哭) 播放 怪獸大哭聲 (Cry)
└─ 0.90s 回歸 anim-idle (待機原位) 回復正常待機/持續瀕死大哭 -

3.1 核心動畫 CSS 定義

- 待機呼吸 (**characterIdle**):

利用 scaleY 與 translateY 的循環微幅變化, 呈現楓之谷式的彈性呼吸感:

```
@keyframes characterIdle {  
  0%, 100% { transform: translateY(0) scaleY(1); }  
  50% { transform: translateY(-5px) scaleY(0.97) scaleX(1.02); }  
}
```

- 斬擊衝刺 (**characterSlash**):

英雄瞬間向右突進 120 像素, 並帶有前傾斜切變 (Skew) 與水平拉伸:

```
@keyframes characterSlash {  
  0% { transform: translateX(0) scaleX(1.1); }  
  30% { transform: translateX(120px) scaleX(1.3) skewX(-10deg); }  
  70% { transform: translateX(110px) scaleX(0.9); }  
  100% { transform: translateX(0) scaleX(1); }  
}
```

- 螢幕受擊震動 (**shake**):

整個戰鬥視窗 (#battle-viewport) 觸發立體位移偏擺:

```
@keyframes shake {  
  10%, 90% { transform: translate3d(-2px, 0, 0); }  
  20%, 80% { transform: translate3d(4px, -2px, 0); }  
  30%, 50%, 70% { transform: translate3d(-6px, 3px, 0); }  
  40%, 60% { transform: translate3d(6px, -3px, 0); }  
}
```

4. 🎧 8-bit 免載入音訊合成引擎 (Web Audio API Retro Engine)

為了保持極致的流暢度並節省伺服器算力 (免載入任何實體音效 mp3), 本系統使用 **HTML5 Web Audio API** 即時合成 8-bit 復古遊戲音效。請 Claude 直接複製以下函式至音訊控制模組中:

4.1 英雄揮刀聲 (Attack Swing Synth)

- 原理: 使用 triangle 三角波, 從低頻快速滑音 (Pitch Sweep) 至高頻, 模擬氣流割裂感。

```
function playSwingSound() {  
  const audioCtx = new (window.AudioContext || window.webkitAudioContext)();
```

```

const osc = audioCtx.createOscillator();
const gain = audioCtx.createGain();

osc.type = 'triangle';
osc.frequency.setValueAtTime(100, audioCtx.currentTime);
osc.frequency.exponentialRampToValueAtTime(1200, audioCtx.currentTime + 0.15);

gain.gain.setValueAtTime(0.3, audioCtx.currentTime);
gain.gain.linearRampToValueAtTime(0.01, audioCtx.currentTime + 0.15);

osc.connect(gain);
gain.connect(audioCtx.destination);

osc.start();
osc.stop(audioCtx.currentTime + 0.15);
}

```

4.2 擊中爆擊聲 (Crit Impact Synth)

- 原理：低頻 sawtooth 鋸齒波模擬撞擊重低音，疊加高頻 sine 正弦波瞬間下降，模擬擊碎金屬或盔甲的清脆聲響。

```

function playHitSound() {
  const audioCtx = new (window.AudioContext || window.webkitAudioContext)();

  // 1. 低頻重擊
  const oscLow = audioCtx.createOscillator();
  const gainLow = audioCtx.createGain();
  oscLow.type = 'sawtooth';
  oscLow.frequency.setValueAtTime(150, audioCtx.currentTime);
  oscLow.frequency.exponentialRampToValueAtTime(40, audioCtx.currentTime + 0.2);

  gainLow.gain.setValueAtTime(0.4, audioCtx.currentTime);
  gainLow.gain.linearRampToValueAtTime(0.01, audioCtx.currentTime + 0.25);

  oscLow.connect(gainLow);
  gainLow.connect(audioCtx.destination);
  oscLow.start();
  oscLow.stop(audioCtx.currentTime + 0.25);

  // 2. 高頻清脆擊打
  const oscHigh = audioCtx.createOscillator();
  const gainHigh = audioCtx.createGain();

```

```

oscHigh.type = 'sine';
oscHigh.frequency.setValueAtTime(900, audioCtx.currentTime);
oscHigh.frequency.setValueAtTime(450, audioCtx.currentTime + 0.05);

gainHigh.gain.setValueAtTime(0.2, audioCtx.currentTime);
gainHigh.gain.linearRampToValueAtTime(0.01, audioCtx.currentTime + 0.15);

oscHigh.connect(gainHigh);
gainHigh.connect(audioCtx.destination);
oscHigh.start();
oscHigh.stop(audioCtx.currentTime + 0.15);
}

```

4.3 怪獸哭泣哀號聲 (Monster Crying Synth)

- 原理：使用溫和的 sine 波，模擬卡通中哭泣時「滑音起伏 (Vibrato & Slide Down)」的顫抖哭腔效果。

```

function playCrySound() {
  const audioCtx = new (window.AudioContext || window.webkitAudioContext)();
  const osc = audioCtx.createOscillator();
  const gain = audioCtx.createGain();

  osc.type = 'sine';
  osc.frequency.setValueAtTime(600, audioCtx.currentTime);
  // 模擬哭腔的波形起伏
  osc.frequency.linearRampToValueAtTime(300, audioCtx.currentTime + 0.4);
  osc.frequency.linearRampToValueAtTime(450, audioCtx.currentTime + 0.8);

  gain.gain.setValueAtTime(0.15, audioCtx.currentTime);
  gain.gain.linearRampToValueAtTime(0.01, audioCtx.currentTime + 0.8);

  osc.connect(gain);
  gain.connect(audioCtx.destination);

  osc.start();
  osc.stop(audioCtx.currentTime + 0.8);
}

```

5. 與原有《開發說明書》系統之串接規範

請 Claude Code 將此戰鬥與視覺層與核心系統 (M1, M3, M4) 進行數據綁定：

1. **金幣扣除 (Coin Deduct)**: 每次點擊「發動攻擊」, 必須向後台發送異步請求。檢查 $\text{points} \geq 5$, 若成功扣除則將 UI 更新為扣除後的點數 (並觸發跳動縮放), 否則不執行攻擊序列, 並讓英雄泡泡顯示提示。
2. **HP 控制閥 (Hp Controller)**: 怪獸血量為 50 點。每次遭受攻擊, 固定扣除由家長配置或預設的傷害比例 (例如每次扣除 15 點)。
3. **大哭臨界值 (Cry Threshold)**: 程式邏輯中, 怪獸 HP 低於或等於 30% 狀態時, UI 上必須:
 - 解除大哭 SVG 的 hidden class。
 - 將怪獸的 div 元素加入大哭專用的顫抖動畫, 並將其整體縮放 (scale) 調整為 0.85 (表示知錯防禦姿態)。
 - 觸發 `playCrySound()`。
4. **消滅狀態 (Eliminated Transition)**: 當 $\text{HP} \leq 0$ 時, 不只觸發受擊, 還要執行 `opacity-0` 與 `rotate-90` 動畫, 並記錄此怪獸為已消滅, 發放家長設定的獎品憑證。